

Módulo 5: Superficie del Servidor

T15: SSRF y APIs Expuestas

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

 *Hacer que el servidor llame a URLs que tú controlas*

Objetivos

- Entender cómo funciona Server-Side Request Forgery (SSRF)
- Explotar un endpoint de VulnApp que hace fetch a URLs arbitrarias
- Acceder a metadatos del servidor de nube (AWS/GCP IMDS) mediante SSRF
- Implementar una allowlist de dominios/IPs como defensa

¿Qué es SSRF?

Server-Side Request Forgery — hacer que el servidor realice peticiones HTTP a URLs que el atacante elige.

```
Atacante → POST /preview {"url": "http://169.254.169.254/latest/meta-data/"}
```

↓

Servidor hace fetch

↓

Devuelve metadatos de la instancia cloud

💀 **Capital One breach (2019):** SSRF al endpoint de metadatos de AWS → acceso a credenciales IAM → 100M registros filtrados.

Targets típicos de SSRF

Target	URL	Qué se obtiene
AWS IMDS	<code>http://169.254.169.254/latest/meta-data/</code>	Credenciales IAM, región, instance-id
GCP Metadata	<code>http://metadata.google.internal/</code>	Service account tokens
Azure IMDS	<code>http://169.254.169.254/metadata/instance</code>	Managed Identity tokens
Servicios internos	<code>http://localhost:6379</code>	Redis, Elasticsearch, DBs internas
Admin panels	<code>http://127.0.0.1:8080/admin</code>	Paneles sin auth (solo red interna)

El patrón vulnerable

```
// Endpoint que hace fetch a una URL proporcionada por el usuario
router.post('/preview', async (req, res) => {
  const { url } = req.body;

  // ⚠️ VULNERABLE: el usuario controla la URL
  const response = await fetch(url);
  const html = await response.text();

  res.json({ preview: html });
});
```

El servidor actúa como proxy — el atacante accede a recursos internos que no alcanzaría directamente.

El fix: allowlist + validación

```
import { URL } from 'url';

const ALLOWED_HOSTS = new Set(['images.unsplash.com', 'cdn.example.com']);
const BLOCKED_IPS = /^(127\.|10\.|172\.([6-9]|2|3[01])\.|192\.168\.|169\.254\.)/;

router.post('/preview', async (req, res) => {
  const { url } = req.body;

  const parsed = new URL(url);

  // ✅ Solo dominios permitidos
  if (!ALLOWED_HOSTS.has(parsed.hostname)) {
    return res.status(400).json({ error: 'Dominio no permitido' });
  }

  // ✅ Bloquear IPs internas
  if (BLOCKED_IPS.test(parsed.hostname)) {
    return res.status(400).json({ error: 'IP interna bloqueada' });
  }

  // ✅ Solo HTTPS
  if (parsed.protocol !== 'https:') {
    return res.status(400).json({ error: 'Solo HTTPS permitido' });
  }

  const response = await fetch(url);
  res.json({ preview: await response.text() });
});
```

APIs expuestas – reducir la superficie

Problema	Solución
Endpoint <code>/debug</code> sin auth	Proteger con <code>requireAuth</code> + <code>requireAdmin</code>
<code>process.env</code> expuesto	Nunca exponer en respuestas HTTP
Endpoints internos accesibles	Network policies, firewall rules
Sin rate limiting	<code>express-rate-limit</code> para prevenir abuso
CORS permisivo (<code>*</code>)	Restringir a orígenes concretos

✓ **Principio:** si un endpoint no necesita ser público, no debe serlo. Mínimo privilegio también aplica a APIs.

Blind SSRF y DNS Rebinding

Blind SSRF: el servidor hace la petición pero no devuelve la respuesta al atacante.

- Detección: observar diferencias de timing o usar out-of-band callbacks (Burp Collaborator)
- Impacto: port scanning interno, triggering acciones en servicios internos

DNS Rebinding:

1. Atacante registra evil.com → resuelve a IP válida (pasa validación)
2. App resuelve evil.com → IP pública (OK)
3. DNS TTL expira, atacante cambia evil.com → 127.0.0.1
4. App hace request → llega a localhost (bypass de la validación)

Defensa contra DNS rebinding:

```
// Resolver DNS ANTES de hacer la petición y validar la IP resuelta
const { address } = await dns.promises.lookup(parsed.hostname);
if (isPrivateIP(address)) return res.status(400).json({ error: 'IP interna' });
```




Lab T15: SSRF en el endpoint de preview

Objetivo: hacer que VulnApp llame a una URL interna mediante SSRF

Setup: `cd VulnApp && npm start`

1. Llama a `POST /preview` con `{"url": "http://localhost:3000/debug"}`
2. Observa que el servidor hace fetch a su propia URL interna
3. Prueba con `{"url": "http://169.254.169.254/latest/meta-data/"}` (simulado)
4. Implementa una allowlist de dominios en `src/routes/` para mitigar SSRF